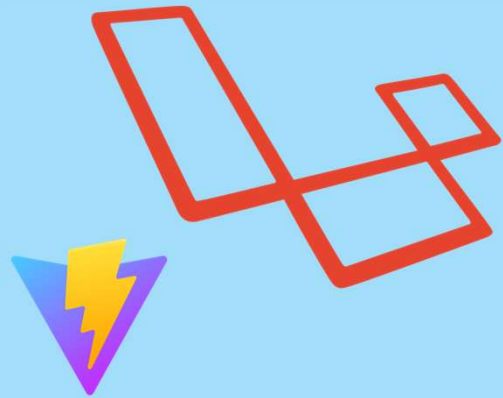




UNIVERSIDAD
**PABLO
OLAVIDE**
SEVILLA



Laravel



Tecnologías Avanzadas de Desarrollo

UD3: Frameworks – Laravel: Usos avanzados y otros

Olga M. Moreno Martín

Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice



Laravel

1. Seeders
2. SQL Views
3. Métodos attach() y detach()
4. Traducciones

Seeders

¿Qué son los seeders?

Los seeders en Laravel son clases que se utilizan para alimentar la base de datos con datos de prueba o predefinidos. Estos datos pueden ser utilizados para probar la funcionalidad de la aplicación, llenar la base de datos con datos iniciales o para cualquier otra necesidad que se requiera.

Los seeders son especialmente útiles cuando se trabaja en equipo, ya que permiten a los desarrolladores tener una base de datos consistente para trabajar, independientemente de las actualizaciones o cambios que se hayan realizado en el código fuente.

Laravel provee un conjunto de herramientas para trabajar con seeders, lo que permite crear, ejecutar y revertir fácilmente la inserción de datos en la base de datos. De esta forma, los desarrolladores pueden mantener la base de datos de la aplicación actualizada y llena de datos de prueba o predefinidos de manera eficiente y controlada.

Seeders

¿Cómo vamos a utilizarlos?

Crea tu seeder utilizando el comando `php artisan make:seeder NombreDelSeeder`.

Abre el archivo del seeder y agrega los datos que quieres insertar en la base de datos.

Para ejecutar un seeder específico, utiliza el comando `php artisan db:seed --class=NombreDelSeeder`, reemplazando "NombreDelSeeder" con el nombre de tu seeder.

Para ejecutar todos los seeders, utiliza el comando `php artisan db:seed`.

Si deseas refrescar la base de datos y volver a ejecutar todos los seeders, utiliza el comando `php artisan migrate:refresh --seed`. Esto eliminará todas las tablas y datos de la base de datos y volverá a crearlas.

También puedes utilizar seeders en conjunto con las migraciones de Laravel para inicializar tu base de datos con datos predefinidos en lugar de tener que insertarlos manualmente.

Seeders

Ejemplo:

```
namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class UsersSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        User::create([
            'name' => 'Juan Perez',
            'email' => 'juan@example.com',
            'password' => bcrypt('12345678')
        ]);

        User::create([
            'name' => 'Maria Gonzalez',
            'email' => 'maria@example.com',
            'password' => bcrypt('65432189')
        ]);
    }
}
```

Views

En Laravel 9, puedes crear SQL views utilizando las migraciones de base de datos. Para crear una vista, primero debes crear una migración que cree la vista en la base de datos.

Crea una migración utilizando el comando `php artisan make:migration create_nombre_de_la_vista_view`.

Abre el archivo de migración que acabas de crear y utiliza el método `DB::statement()` para crear la vista.

En este ejemplo, se crea una vista llamada "nombre_de_la_vista" que selecciona ciertas columnas de dos tablas diferentes.

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Support\Facades\DB;

class CreateNombreDeLaVistaView extends Migration
{
    public function up()
    {
        DB::statement('CREATE VIEW nombre_de_la_vista AS SELECT
        columna_1, columna_2, ... FROM tabla_1 JOIN tabla_2 ON tabla_1.
        columna = tabla_2.columna;');
    }

    public function down()
    {
        DB::statement('DROP VIEW IF EXISTS nombre_de_la_vista;');
    }
}
```

Views

Una vez creada la migración ejecútala, utilizando el comando `php artisan migrate`.

Una vez que hayas creado la migración y la hayas ejecutado, la vista estará disponible en la base de datos y podrás utilizarla en tu aplicación Laravel como cualquier otra tabla.

Para utilizar la vista en un modelo, simplemente define el nombre de la vista en la propiedad `table` del modelo. Por ejemplo:

De esta manera, podrás utilizar la vista como si fuera una tabla más en la base de datos.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class NombreDelModelo extends Model
{
    protected $table = 'nombre_de_la_vista';

    // ...
}
```


Métodos attach y detach

En Laravel 11, `attach()` y `detach()` son métodos que se utilizan para relacionar y desvincular modelos en relaciones de muchos a muchos.

Supongamos que tienes una relación muchos a muchos entre dos modelos, por ejemplo, la relación entre usuarios y roles, donde un usuario puede tener muchos roles y un rol puede ser asignado a muchos usuarios.

En este caso, tendrías tres tablas en tu base de datos: la tabla `users`, la tabla `roles` y la tabla pivote `role_user`.

Para agregar un rol a un usuario, puedes utilizar el método `attach()` en el modelo del usuario, como se muestra a continuación:

```
$user = User::find(1);  
$user->roles()->attach($roleId);
```

En este ejemplo, `$roleId` es el id del rol que deseas agregar al usuario. El método `attach()` añadirá la relación entre el usuario y el rol en la tabla pivote `role_user`.

Métodos attach y detach

Para desvincular un rol de un usuario, puedes utilizar el método detach() en el modelo del usuario, como se muestra a continuación:

```
$user = User::find(1);  
$user->roles()->detach($roleId);
```

En este ejemplo, \$roleId es el id del rol que deseas desvincular del usuario. El método detach() eliminará la relación entre el usuario y el rol de la tabla pivote role_user.

También es posible agregar varios roles a la vez utilizando el método attach() pasando un array de ids, como se muestra a continuación:

```
$user = User::find(1);  
$user->roles()->attach([$roleId1, $roleId2, $roleId3]);
```

De esta forma, podrás agregar varios roles al usuario en una sola llamada al método attach().

Es importante tener en cuenta que, al utilizar attach() y detach(), estás modificando la base de datos. Por lo tanto, siempre debes tener cuidado al utilizar estos métodos y asegurarte de que la información que estás agregando o eliminando es la correcta.

Traducciones



En Laravel 9, puedes utilizar las traducciones para mostrar contenido en diferentes idiomas. Laravel proporciona una variedad de herramientas para trabajar con traducciones y hacer que tu aplicación sea multilingüe.

- **Agrega los archivos de traducción:** Laravel utiliza archivos de traducción para almacenar el contenido en diferentes idiomas. Estos archivos se encuentran en el directorio `resources/lang` y se organizan en subdirectorios que representan los idiomas compatibles. Por ejemplo, si tu aplicación admite inglés y español, deberías tener dos subdirectorios: `en` y `es`.

Traducciones

Crea archivos de traducción: Dentro de cada subdirectorio de idioma, crea un archivo de traducción que contenga las cadenas de texto que deseas traducir. Por ejemplo, si tienes una vista que muestra el mensaje "Welcome to my website", podrías agregar una entrada de traducción para ese mensaje en el archivo [resources/lang/en/messages.php](#)

```
<?php
return [
    'welcome_message' => 'Welcome to my website',
];
```

Luego, agrega la misma entrada de traducción en el archivo [resources/lang/es/messages.php](#), pero con la traducción en español:

```
<?php
return [
    'welcome_message' => 'Bienvenido a mi sitio web',
];
```

Traducciones

- **Utiliza las traducciones en tu aplicación:** Una vez que has agregado los archivos de traducción y las entradas de traducción, puedes utilizar las traducciones en tu aplicación utilizando la función `trans()` o la directiva `@lang` en las vistas de Blade. Por ejemplo:

```
// Utilizando la función trans()  
echo trans('messages.welcome_message');  
  
// Utilizando la directiva @lang en una vista de Blade  
@lang('messages.welcome_message')
```

En ambos casos, Laravel buscará la entrada de traducción correspondiente en el archivo de traducción adecuado en función del idioma actual de la aplicación. Si no se encuentra la entrada de traducción correspondiente en el archivo de traducción, se utilizará el valor predeterminado especificado en el primer parámetro de la función `trans()` o la directiva `@lang`.

Traducciones

Para cambiar el idioma de la aplicación, debes utilizar la función `app()->setLocale($locale)` donde `$locale` es el código del idioma que deseas utilizar. Por ejemplo:

```
app()->setLocale('es');
```

De esta forma, Laravel buscará las entradas de traducción en el archivo de traducción correspondiente al idioma español.

¿Preguntas?

Olga M. Moreno Martín
Olga M. Moreno Martín

